

alone boards from other manufacturers. Initially you only need to understand the basic operation of the system – the main purpose of the board, how the data flows through the board and the input/outputs available.

Once you're clear about the basics it will time to dig deeper. You must then go through the hardware specification which contain detailed schematic of components and peripheral available in the board. The best way to start is to go through the 'user guide' or 'programmers manual' that came along with the board. This usually contains a detailed description of each component/peripheral on the board, it's hardware specifications and how it can be programmed. In case the board is custom designed for you then you'll find a more pedantic documentation may be geared towards hardware engineers. The next step is understand how memory is managed by the system. Processors use a set of two buses to interact with the memory - the address bus and the data bus. To read or write to an address the processor selects an address on the address bus which is then passed to the address decoder which translates the location's logical address to the physical address. There is also a control bus which is used to control the bus signals like read, write and chip-select/chip-enable(CS/CE).

Next step is getting to know the processor. This will not take much time if you're going to do all your programming in a high-level language like C. You'll have to take some time and understand how the peripherals of your processor work. If you're using C, most processors will look and act pretty much in the same way. However, if you're planning to program them using assembly, then it's advised that you familiarise yourself with the processor's register architecture and it's instruction set. Once again the databook will be your best friend here, processor user guides/databooks are usually well written as databooks go.

The final step as far as understanding the hardware is concerned is to understand the working of all peripherals and devices that will be used in your system. These devices communicate with the processor by interrupts and I/O or memory-mapped registers. Depending on your application, the device(s) might include LCD or keyboard controllers, analog-to-digital (A/D) converters, network interface chips, or custom application-specific integrated circuits (ASICs). Procure a data-sheet for each component, in the early stages of the project you only need to understand the basics of each of them.

A good programming practice is to collect all the data sheets/user manuals in one place in either physical or electronic form. This way you can